

Performance Overhead of JWT-Based Authentication in REST APIs

A Comparative Experimental Study of Authenticated and Unauthenticated API Performance

Amirbek Erkinov

Independent Researcher

Tashkent, Uzbekistan

September 2026

Contents

Abstract

1. Introduction

1.1 Background

1.2 Research Problem

1.3 Research Aim

1.4 Research Questions

1.5 Research Hypothesis

1.6 Research Contribution

2. Related Work and Background

2.1 REST API Security

2.2 JWT Authentication

2.3 Performance Metrics

2.3.1 Throughput

2.3.2 Average Latency

2.3.3 P99 Latency

3. Methodology

3.1 Experimental Design

3.2 API Implementation

3.2.1 Unauthenticated Baseline

3.2.2 JWT-Authenticated API

3.3 Benchmark Configuration

3.4 Measurements and Analysis

3.5 Experimental Environment

4. Results

4.1 Throughput Results

4.2 Average Latency Results

4.3 P99 Latency Results

4.4 Summary of Results

5. Discussion

5.1 Interpretation of Findings

5.2 Effect of Increasing Concurrency

5.3 Research Questions and Hypothesis

6. Limitations

7. Conclusion

References

Appendix A – Research Repository

Appendix B – Benchmark Results

Appendix C – Analysis Code

Abstract

In order to protect confidential and sensitive information, REST APIs that handle protected resources commonly use authentication mechanisms, which may add processing overhead to each request. The main goal of this study is to examine the performance overhead of JSON Web Tokens (JWT)-based authentication compared with an equivalent unauthenticated approach. An unauthenticated baseline and JWT-protected implementations providing the same `/profile` functionality were implemented using Node.js and an Express API. Autocannon was used for benchmarking purposes at 10, 25, 50 and 100 concurrent connections, with five 10-second repetitions for each configuration. Throughput, average latency and P99 latency were measured and compared. The JWT implementation achieved approximately 3,814-3,834 requests per second, while the baseline achieved 18,256-18,710 requests per second. This represents approximately a 79% reduction in throughput under the same conditions. Furthermore, average latency for the JWT-based implementation was consistently higher, increasing from 2.242 ms at 10 connections to 25.680 ms at 100 connections. Additionally, P99 latency was 37.8 ms for the JWT-based implementation compared with 6.4 ms for the baseline. The results support the hypothesis, meaning the JWT-based implementation introduces measurable request processing overhead. However, the findings are limited and specific to the localhost environment, hardware, and workload and should not be generalised to all JWT-based implementations.

1. Introduction

Representational State Transfer architectural style widely used as a communication means between software components. REST was introduced by Fielding (2000) as an architectural style for distributed hypermedia systems, with an emphasis on scalability, general interfaces and independent deployment of components. Today, APIs are an important part of web, mobile and distributed applications, where they provide access to application functionality and data.

Authentication mechanisms are used to determine whether a user has the right to access protected resources provided by an API. Therefore, security aspects of public APIs are an important consideration in modern software systems. The Open Worldwide Application Security Project (OWASP) identifies broken authentication as one of the major API security risks, highlighting the importance of correctly implementing authentication mechanisms and protecting authentication credentials and tokens (OWASP, 2023).

JSON Web Token (JWT) is one mechanism that can be used to support authentication in web APIs. A JWT is a compact, URL-safe representation of claims that can be transmitted between parties. JWTs can be digitally signed or protected using a message authentication code (MAC), which allows a receiving application to verify the integrity of the token and its claims (Jones, Bradley and Sakimura, 2015). In an API, a client can include a JWT with a request to a protected endpoint, after which the server can verify the token before processing the request.

Despite the important security function provided by JWT tokens, they also introduce additional processing compared with an equivalent request that does not require authentication verification. For a JWT-protected request, the server must retrieve the token from the request, verify it and process the associated claims before the protected resource can be returned. This additional processing may affect API performance, particularly when an application is required to handle a large number of concurrent requests. Therefore, understanding the performance implications of authentication is relevant when evaluating the trade-off between API security and system performance.

This study investigates the performance overhead associated with JWT-based authentication by comparing a JWT-protected REST API with an equivalent unauthenticated baseline. Two implementations were developed using Node.js and Express, with both providing the same `/profile` functionality. The implementations were benchmarked under increasing levels of

concurrent connections in order to examine whether the addition of JWT authentication affected throughput and request latency.

The main research question of this study is:

What performance overhead does JWT-based authentication introduce when processing authenticated REST API requests compared with an equivalent unauthenticated baseline?

A secondary research question is:

How does the overhead vary under increasing concurrent request loads?

The study hypothesises that **JWT-based authentication introduces measurable request processing overhead compared with an unauthenticated baseline**. To investigate this hypothesis, throughput, average latency and P99 latency were measured at 10, 25, 50 and 100 concurrent connections. Each configuration was tested using five repeated 10-second benchmark runs, allowing the performance of the two implementations to be compared under the same experimental conditions.

2.1 REST API security

Commonly, the resources and functionality of an application are provided through a defined architecture in REST APIs. If endpoints contain information or functionality which should not be publicly accessible, APIs need to implement mechanisms such as access control. Authentication is one of the mechanisms used to establish the identity of a client before allowing access to protected resources (OWASP, 2023).

In an authenticated API call, the credentials sent by the client must be obtained and validated before continuing operations. If credentials are incorrect or missing, the request should be rejected, rather than providing access to protected resources. This authentication process therefore introduces additional processing compared with an equivalent request that does not require authentication.

APIs frequently expose resources directly to clients and may provide access to sensitive application functionality or data; therefore, authentication is particularly relevant to API security. OWASP identifies authentication-related weaknesses as an important category of API security risk, highlighting the need for authentication mechanisms to be implemented correctly (OWASP, 2023).

From a performance perspective, authentication represent an additional stage in the request-processing process. Consequently, comparing an authenticated endpoint with an equivalent unauthenticated endpoint proved a way to measure the performance impact associated with adding authentication. This forms the basis for the experimental comparison conducted in this study.

2.2 JWT Authentication

JSON Web Token (JWT) is a URL-safe, compact format for representing claims between two parties. In order to verify the integrity of the token and its claims, JWT can be digitally signed or protected using a message authentication code (Jones, Bradley, and Sakimura, 2015).

There are 3 main components in a JWT: a header, a payload and a signature. While the information about the token is described in the header, payload contains claims associated with this token. The signature is used to verify the integrity of token after it issued (Jones, Bradley, and Sakimura, 2015).

During authentication process, user can include JWT token during HTTP/HTTPS request in order to access protected resources or functionality. Appropriate signature or cryptography key used to verify the signature, once it has been extracted from request. If the token valid, the claim contained within it can be used by application to identify the authenticated user process the request. Server can reject request if token invalid or cannot be verified.

JWT authentication therefore introduces additional processing into the request-handling path. Compared with an unauthenticated request, the server must perform operations such as retrieving the token, verifying its signature and processing its claims before the protected endpoint can be accessed. The effect of these additional operations on API performance is the focus of the experimental evaluation in this study.

2.3 Performnace metrics

API performance can be evaluated using measurments such as how efficiently server can processes requests and how quickly it can respond to clients. This study uses throughput, average latency, and P99 latency because these metrics provide complementary view.

Throughput represents the number of requests that an application can process within a given period. In this study, throughput is measured in requests per second (req/s). A higher throughput indicates that the implementation is able to process more requests under the tested conditions.

Latency represents the time required to process a request and return a response to the client. In this study, latency is measured in milliseconds (ms). Average latency provides an overall measure of the response time observed during each benchmark run.

However, an average value does not fully describe the slower requests experienced during a benchmark. Therefore, this study also considers **P99 latency**, which represents the 99th percentile of measured request latency. This metric provides information about the upper end of the latency distribution and can reveal slower responses that may not be apparent from the average latency alone.

These three metrics were selected because they allow the performance effects of JWT authentication to be examined from different perspectives. Throughput measures the amount of work processed by the API, while average latency measures general response time and P99 latency provides information about higher-latency requests. Together, these measurements provide a broader basis for comparing the authenticated and unauthenticated implementations.

3. Methodology

3.1 Experimental Design

A controlled experimental design was used to compare the performance of an unauthenticated REST API with an equivalent API protected using JWT authentication. Two separate implementations were developed using Node.js and Express. Both implementations provided the same `/profile` endpoint and returned an equivalent response structure. The primary difference between the implementations was the presence of JWT authentication in the protected version.

The APIs were evaluated under four levels of concurrent connections: 10, 25, 50 and 100. Each configuration was executed five times for a duration of 10 seconds. This resulted in 40 individual benchmark runs across the two implementations and four concurrency levels.

3.2 API Implementation

Two REST API implementations were developed using Node.js and Express. The first implementation served as the unauthenticated baseline, while the second implemented JWT-based authentication. Both applications provided the same `/profile` endpoint and returned an equivalent response containing user information. This was intended to minimise differences between the two implementations that were unrelated to authentication.

The baseline API processed requests to `/profile` directly through the Express route without performing an authentication check. In contrast, the JWT implementation placed an authentication middleware before the `/profile` route. The middleware retrieved the JWT from the HTTP `Authorization` header and verified it using the `jsonwebtoken` library and a secret stored in an environment variable. When verification was successful, the decoded token information was attached to the request and the request was passed to the `/profile` route. Requests containing a missing or invalid token were rejected with an HTTP 401 response.

A `/login` endpoint was also implemented to generate a valid JWT for the benchmark. However, token generation and login processing were excluded from the performance measurements. The benchmark instead used an already-issued valid token so that the experiment focused on the processing overhead associated with authenticated access to the protected resource.

The two implementations therefore followed the same general request-response process, with JWT extraction and verification representing the additional processing introduced in the authenticated implementation.

3.3 Benchmark Configuration

Autocannon was used to evaluate the performance of the two API implementations. The `/profile` endpoint was benchmarked at four levels of concurrency: 10, 25, 50 and 100 concurrent connections. Each configuration was executed for 10 seconds and repeated five times for both the baseline and JWT implementations.

Pipelining was set to 1 so that each connection maintained one outstanding request at a time. In this experiment, the concurrency value represents the number of simultaneous HTTP connections maintained during a benchmark rather than the total number of requests generated. Requests were continuously issued during each 10-second test period.

For the JWT implementation, an already-issued valid JWT was included in the `Authorization` header of each request. The baseline requests did not contain authentication information. The benchmark was executed against the same `/profile` functionality in both implementations.

The combination of two API configurations, four concurrency levels and five repetitions resulted in a total of 40 benchmark runs. The repeated measurements were used to calculate descriptive statistics and compare the performance of the two implementations.

3.4 Measurements and Analysis

Three performance metrics were collected from each benchmark run: throughput, average latency and P99 latency. Throughput was recorded in requests per second, while latency measurements were recorded in milliseconds.

For each combination of API configuration and concurrency level, five benchmark repetitions were performed. The five observations were grouped according to the API configuration and concurrency level, after which the arithmetic mean and standard deviation were calculated. The mean was used to represent the average observed performance, while the standard deviation was used to describe the variation between repeated runs.

The resulting measurements were compared between the baseline and JWT implementations at each concurrency level. The results were also visualised using graphs showing throughput, average latency and P99 latency against concurrent connections. Error bars representing one standard deviation were included where appropriate to show variation between repeated measurements.

The analysis was performed using Python. The benchmark results were stored in JSON format and processed using a separate analysis script. This separation allowed the raw benchmark data to remain unchanged while the statistical calculations and visualisations were generated from the recorded results.

3.5 Experimental Environment

The experiment was conducted in a local development environment using a 16-inch MacBook Pro with an Apple M3 Pro processor and 18 GB of memory. The system was running macOS Tahoe 26.5.2. Both the API server and the Autocannon benchmarking client were executed on the same computer and communicated through the local host interface.

The API implementations were developed using Node.js version 23.2.0 and Express version 5.2.1. JWT authentication was implemented using `jsonwebtoken` version 9.0.3, while `dotenv` version 18.0.3 was used to load environment variables. Benchmarking was performed using Autocannon version 8.0.0, and Python version 3.12.7 was used for statistical analysis and visualisation. The exact package versions used by the project are recorded in the project's package lock file.

Component	Specification
Computer	MacBook Pro 16-inch (November 2023)
Processor	Apple M3 Pro
Memory	18 GB
Operating system	macOS Tahoe 26.5.2
Node.js	23.2.0
Python	3.12.7
Express	5.2.1
jsonwebtoken	9.0.3
Autocannon	8.0.0
dotenv	18.0.3

Since both the server and benchmarking client were executed on the same computer, the results represent the behaviour of the tested implementations under the specified local hardware and software conditions. Therefore, the findings should not be interpreted as universal performance measurements for JWT authentication.

4. Results

4.1 Throughput Results

The throughput results show a substantial difference between the unauthenticated baseline and the JWT-authenticated implementation across all tested concurrency levels. The mean throughput values for both implementations are presented in Table 1.

Table 1. Mean throughput at each concurrency level

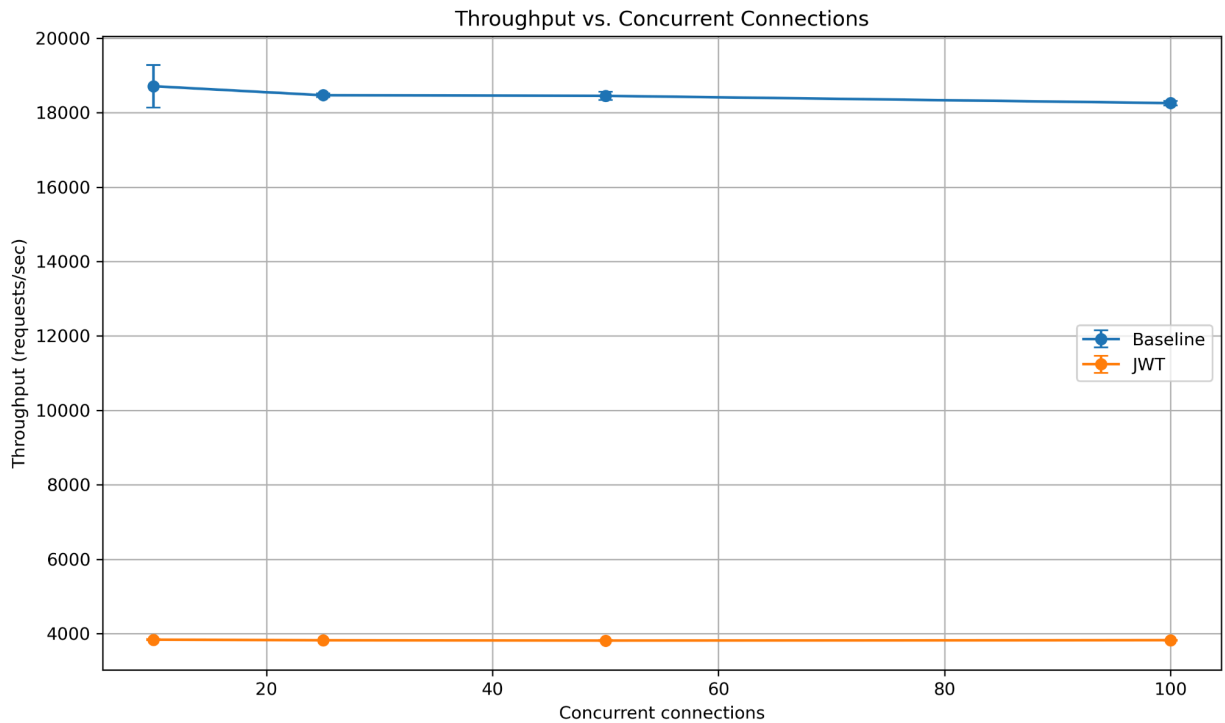
Concurrent connections	Baseline (requests/s)	JWT (requests/s)
10	18,710.14	3,834.39
25	18,467.06	3,822.31
50	18,451.06	3,814.35
100	18,256.15	3,824.20

The baseline achieved a mean throughput between 18,256.15 and 18,710.14 requests per second across the tested concurrency levels. In comparison, the JWT implementation achieved between 3,814.35 and 3,834.39 requests per second. The JWT implementation therefore produced approximately 79% lower throughput than the baseline at each concurrency level.

The baseline throughput decreased slightly as concurrency increased, from 18,710.14 requests per second at 10 connections to 18,256.15 requests per second at 100 connections. The JWT throughput remained relatively stable, with only small differences between the four tested

concurrency levels. Figure 1 illustrates the throughput measurements and the variation between repeated runs.

Figure 1. Throughput vs. concurrent connections. Error bars represent ± 1 standard deviation across five benchmark repetitions.



4.2 Average Latency Results

The average latency results show that both implementations experienced increased response times as the number of concurrent connections increased. However, the JWT implementation recorded higher average latency at every tested concurrency level.

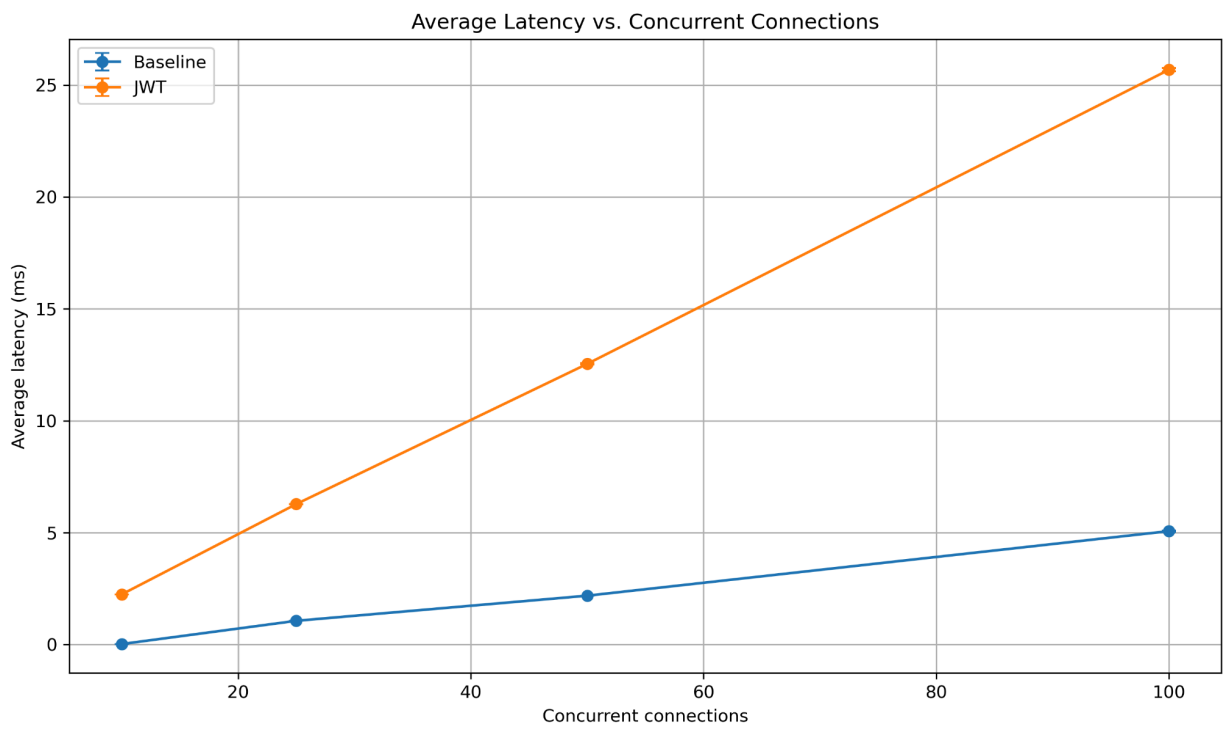
Table 2. Mean average latency at each concurrency level

Concurrent connections	Baseline (ms)	JWT (ms)
10	0.022	2.242
25	1.060	6.276
50	2.182	12.538
100	5.066	25.680

At 10 concurrent connections, the baseline recorded a mean latency of 0.022 ms, compared with 2.242 ms for the JWT implementation. At 25 connections, the mean latency increased to 1.060 ms for the baseline and 6.276 ms for JWT. At 50 connections, the corresponding values were 2.182 ms and 12.538 ms. At the highest tested concurrency level of 100 connections, the baseline recorded a mean latency of 5.066 ms, while the JWT implementation recorded 25.680 ms.

The difference between the two implementations became more pronounced as concurrency increased. At 100 concurrent connections, the mean latency of the JWT implementation was approximately five times higher than that of the baseline. The results are shown in Figure 2.

Figure 2. Average latency vs. concurrent connections. Error bars represent ± 1 standard deviation across five benchmark repetitions.



4.3 P99 Latency Results

P99 latency was analysed to examine the higher end of the latency distribution. The mean P99 latency values for both implementations are presented in Table 3.

Table 3. Mean P99 latency at each concurrency level

Concurrent connections	Baseline (ms)	JWT (ms)
10	1.0	4.0
25	2.0	12.0
50	4.4	25.0
100	6.4	37.8

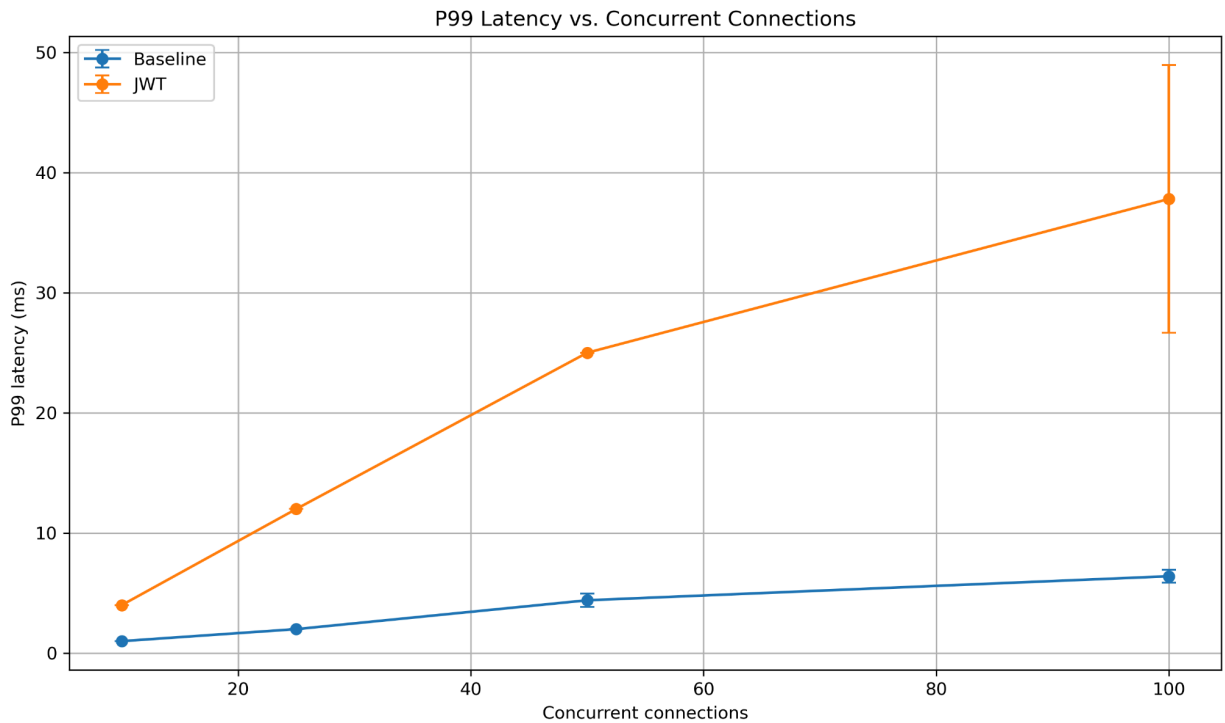
The P99 latency of the baseline increased from 1.0 ms at 10 concurrent connections to 6.4 ms at 100 connections. The JWT implementation showed a larger increase, from 4.0 ms at 10 connections to 37.8 ms at 100 connections.

At 25 concurrent connections, the mean P99 latency was 12.0 ms for the JWT implementation compared with 2.0 ms for the baseline. At 50 connections, the corresponding values were 25.0 ms and 4.4 ms. At 100 connections, the JWT implementation reached a mean P99 latency of 37.8 ms, compared with 6.4 ms for the baseline.

The JWT implementation also showed greater variation in P99 latency at 100 concurrent connections. The five JWT benchmark runs produced P99 values of 30 ms, 30 ms, 29 ms, 50

ms and 50 ms, resulting in a mean of 37.8 ms. This variation is represented by the larger error bar at 100 connections in Figure 3.

Figure 3. P99 latency vs. concurrent connections. Error bars represent ± 1 standard deviation across five benchmark repetitions.



4.4 Summary of Results

Overall, the results show a consistent performance difference between the unauthenticated baseline and the JWT-authenticated implementation under the tested conditions. The JWT implementation achieved approximately 79% lower throughput than the baseline across all four concurrency levels.

Average latency was also consistently higher for the JWT implementation. Its mean latency increased from 2.242 ms at 10 concurrent connections to 25.680 ms at 100 connections, while the baseline increased from 0.022 ms to 5.066 ms over the same range.

A similar pattern was observed for P99 latency. The JWT implementation increased from a mean P99 latency of 4.0 ms at 10 connections to 37.8 ms at 100 connections, compared with 1.0 ms and 6.4 ms respectively for the baseline.

Taken together, the throughput, average latency and P99 latency measurements demonstrate a measurable performance difference between the two implementations under the experimental conditions. The possible reasons for these differences and their implications are discussed in the following section.

5.1 Interpretation of Findings

The experimental results demonstrate a consistent performance difference between the unauthenticated baseline and the JWT-authenticated implementation. Across all tested concurrency levels, the JWT implementation achieved substantially lower throughput and higher average and P99 latency. The difference was observed at both low and high concurrency levels, indicating that the additional authentication processing was associated with measurable changes in the performance of the tested implementation.

The JWT implementation introduces additional processing into the request-handling path that is not present in the baseline. Before the `/profile` endpoint can process an authenticated request, the server must retrieve the token from the `Authorization` header, verify the token and process the resulting claims. The baseline implementation does not perform these authentication operations. The observed performance difference is therefore consistent with the additional processing introduced by the JWT implementation.

However, the experiment measured the complete HTTP request-processing path rather than the isolated execution time of the `jwt.verify()` operation. Therefore, the results cannot be used to determine the exact proportion of the measured overhead caused by JWT cryptographic verification alone. The findings instead describe the performance difference between the two complete API implementations under the tested conditions.

5.2 Effect of Increasing Concurrency

Increasing the number of concurrent connections affected the latency of both implementations more noticeably than their throughput. Baseline throughput decreased only slightly, from 18,710.14 requests per second at 10 connections to 18,256.15 requests per second at 100 connections. JWT throughput also remained relatively stable, ranging from 3,814.35 to 3,834.39 requests per second across the tested concurrency levels.

In contrast, average latency increased substantially as concurrency increased. For the baseline, mean latency increased from 0.022 ms at 10 connections to 5.066 ms at 100 connections. For the JWT implementation, mean latency increased from 2.242 ms to 25.680 ms over the same range.

P99 latency showed a similar pattern. The JWT implementation increased from a mean P99 latency of 4.0 ms at 10 connections to 37.8 ms at 100 connections, while the baseline increased from 1.0 ms to 6.4 ms. This indicates that higher concurrency was associated with increased response times, particularly for the JWT implementation. The larger variation in JWT P99 latency at 100 connections also suggests that the higher-concurrency condition produced less consistent tail-latency measurements in the tested environment.

5.3 Research Questions and Hypothesis

The findings provide evidence relevant to both research questions. For the main research question, the JWT-authenticated implementation demonstrated measurable performance differences compared with the unauthenticated baseline. Throughput was approximately 79% lower for the JWT implementation across the tested concurrency levels, while both average and P99 latency were consistently higher.

The secondary research question considered how the overhead changed as concurrent connections increased. The throughput difference remained relatively consistent across the tested levels, whereas latency increased considerably for both implementations and more strongly for the JWT implementation. Mean JWT latency increased from 2.242 ms at 10 connections to 25.680 ms at 100 connections. P99 latency also increased from 4.0 ms to 37.8 ms over the same range.

These findings support the hypothesis that the JWT-based implementation introduces measurable request-processing overhead compared with the unauthenticated baseline. However, the results should be interpreted within the scope of the experimental design. They describe the behaviour of the specific Node.js and Express implementations tested on the specified local environment and do not establish that all JWT-based systems will experience the same level of performance overhead.

6. Limitations

Several limitations should be considered when interpreting the findings of this study. First, the experiment was conducted in a local environment, with both the API server and benchmarking client running on the same computer. As a result, the measurements do not include network latency or other effects that may occur when an API is accessed over a real network.

Second, the experiment was conducted using a single hardware and software environment and a single Node.js and Express implementation. The observed performance differences may therefore vary when different hardware, operating systems, programming frameworks or JWT libraries are used. The results should consequently not be generalised to all JWT-based API implementations.

The API used in the experiment was also intentionally simple and contained no database operations or complex application logic. This allowed the authentication component to be compared under controlled conditions, but real-world APIs may contain additional processing that could change the relative impact of authentication.

The benchmark used four concurrency levels and five repetitions of 10 seconds for each configuration. Although repeated measurements provide information about variation between runs, a larger range of concurrency levels and longer test durations could provide additional information about system behaviour under sustained and higher workloads.

Another limitation is that the benchmark client and server shared the same computer. The resources used by Autocannon may therefore interact with the resources available to the API server. In addition, the benchmark measured the complete HTTP request-processing path rather than isolating the execution time of JWT verification. Consequently, the experiment cannot determine the exact proportion of the measured performance difference attributable specifically to the cryptographic verification operation.

Finally, the benchmark configurations were executed in a fixed order, with the baseline tested before the JWT implementation. Although the same general conditions were used for both implementations, changing system conditions during the experiment could potentially introduce an order effect. Future experiments could reduce this limitation by randomising the order of benchmark runs and repeating the experiment across multiple independent environments.

7. Conclusion

This study investigated the performance overhead associated with JWT-based authentication by comparing a JWT-protected REST API with an equivalent unauthenticated baseline. Both implementations were developed using Node.js and Express and were evaluated at 10, 25, 50 and 100 concurrent connections using repeated Autocannon benchmark runs.

The results demonstrated a consistent performance difference between the two implementations under the tested conditions. The JWT implementation achieved approximately 79% lower throughput than the baseline across the tested concurrency levels. Average latency was also higher for the JWT implementation and increased from 2.242 ms at 10 concurrent connections to 25.680 ms at 100 connections. P99 latency showed a similar pattern, increasing from 4.0 ms to 37.8 ms for the JWT implementation.

The findings support the hypothesis that adding JWT authentication to the tested API introduced measurable request-processing overhead. The results also indicate that increasing concurrency was associated with substantially higher latency, particularly for the JWT implementation.

However, the findings are specific to the implementation, workload and local experimental environment used in this study. The experiment measured the performance of a complete JWT-authenticated request path rather than the isolated cost of JWT verification, and therefore the results should not be interpreted as a universal measure of JWT performance. Further research could extend the experiment by testing different hardware environments, JWT algorithms, authentication libraries, larger workloads and more complex API operations.

Appendix A – Research Repository

The complete source code, benchmark implementation, raw experimental results and data analysis scripts used in this study are available in the project's GitHub repository:

GitHub Repository: <https://github.com/amir2501/jwt-research>

The repository contains the following:

- REST API implementations for the baseline and JWT-authenticated configurations
- JWT authentication middleware
- Benchmarking script
- Raw benchmark results
- Python analysis scripts
- Generated performance figures
- Project documentation and experimental configuration

Appendix B – Benchmark Results

```
[
  {
    "configuration": "baseline",
    "connections": 10,
    "duration": 10,
    "requests": 19727.28,
    "latency": {
      "average": 0.02,
      "median": 0,
      "p97_5": 0,
      "p99": 1
    },
    "errors": 0,
    "timeouts": 0,
    "non2xx": 0,
    "run": 1
  },
  {
    "configuration": "jwt",
    "connections": 10,
    "duration": 10,
    "requests": 3841.55,
    "latency": {
      "average": 2.25,
      "median": 2,
      "p97_5": 4,
      "p99": 4
    },
    "errors": 0,
    "timeouts": 0,
    "non2xx": 0,
    "run": 1
  },
  {
    "configuration": "baseline",
    "connections": 10,
    "duration": 10,
    "requests": 18422.55,
    "latency": {
      "average": 0.02,
      "median": 0,
      "p97_5": 0,
      "p99": 1
    },
    "errors": 0,
    "timeouts": 0,
    "non2xx": 0,
```

```
"run": 2
},
{
  "configuration": "jwt",
  "connections": 10,
  "duration": 10,
  "requests": 3809.37,
  "latency": {
    "average": 2.24,
    "median": 2,
    "p97_5": 4,
    "p99": 4
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 2
},
{
  "configuration": "baseline",
  "connections": 10,
  "duration": 10,
  "requests": 18449.6,
  "latency": {
    "average": 0.02,
    "median": 0,
    "p97_5": 0,
    "p99": 1
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 3
},
{
  "configuration": "jwt",
  "connections": 10,
  "duration": 10,
  "requests": 3820.28,
  "latency": {
    "average": 2.24,
    "median": 2,
    "p97_5": 4,
    "p99": 4
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 3
}
```

```
},
{
  "configuration": "baseline",
  "connections": 10,
  "duration": 10,
  "requests": 18479.28,
  "latency": {
    "average": 0.03,
    "median": 0,
    "p97_5": 0,
    "p99": 1
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "jwt",
  "connections": 10,
  "duration": 10,
  "requests": 3850.46,
  "latency": {
    "average": 2.24,
    "median": 2,
    "p97_5": 4,
    "p99": 4
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "baseline",
  "connections": 10,
  "duration": 10,
  "requests": 18472,
  "latency": {
    "average": 0.02,
    "median": 0,
    "p97_5": 0,
    "p99": 1
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 5
},
}
```

```
{
  "configuration": "jwt",
  "connections": 10,
  "duration": 10,
  "requests": 3850.28,
  "latency": {
    "average": 2.24,
    "median": 2,
    "p97_5": 4,
    "p99": 4
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 5
},
{
  "configuration": "baseline",
  "connections": 25,
  "duration": 10,
  "requests": 18441.46,
  "latency": {
    "average": 1.06,
    "median": 1,
    "p97_5": 2,
    "p99": 2
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 1
},
{
  "configuration": "jwt",
  "connections": 25,
  "duration": 10,
  "requests": 3828.64,
  "latency": {
    "average": 6.28,
    "median": 6,
    "p97_5": 12,
    "p99": 12
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 1
},
{
```

```
"configuration": "baseline",
"connections": 25,
"duration": 10,
"requests": 18418.19,
"latency": {
  "average": 1.06,
  "median": 1,
  "p97_5": 2,
  "p99": 2
},
"errors": 0,
"timeouts": 0,
"non2xx": 0,
"run": 2
},
{
  "configuration": "jwt",
  "connections": 25,
  "duration": 10,
  "requests": 3819.91,
  "latency": {
    "average": 6.28,
    "median": 6,
    "p97_5": 12,
    "p99": 12
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 2
},
{
  "configuration": "baseline",
  "connections": 25,
  "duration": 10,
  "requests": 18442.91,
  "latency": {
    "average": 1.06,
    "median": 1,
    "p97_5": 2,
    "p99": 2
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 3
},
{
  "configuration": "jwt",
```



```
"connections": 25,
"duration": 10,
"requests": 3811,
"latency": {
  "average": 6.28,
  "median": 6,
  "p97_5": 12,
  "p99": 12
},
"errors": 0,
"timeouts": 0,
"non2xx": 0,
"run": 3
},
{
  "configuration": "baseline",
  "connections": 25,
  "duration": 10,
  "requests": 18493.82,
  "latency": {
    "average": 1.06,
    "median": 1,
    "p97_5": 2,
    "p99": 2
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "jwt",
  "connections": 25,
  "duration": 10,
  "requests": 3826.28,
  "latency": {
    "average": 6.27,
    "median": 6,
    "p97_5": 12,
    "p99": 12
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "baseline",
  "connections": 25,
```

```
"duration": 10,
"requests": 18538.91,
"latency": {
  "average": 1.06,
  "median": 1,
  "p97_5": 2,
  "p99": 2
},
"errors": 0,
"timeouts": 0,
"non2xx": 0,
"run": 5
},
{
  "configuration": "jwt",
  "connections": 25,
  "duration": 10,
  "requests": 3825.73,
  "latency": {
    "average": 6.27,
    "median": 6,
    "p97_5": 12,
    "p99": 12
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 5
},
{
  "configuration": "baseline",
  "connections": 50,
  "duration": 10,
  "requests": 18488,
  "latency": {
    "average": 2.18,
    "median": 2,
    "p97_5": 3,
    "p99": 4
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 1
},
{
  "configuration": "jwt",
  "connections": 50,
  "duration": 10,
```

```
"requests": 3838.64,
"latency": {
  "average": 12.47,
  "median": 12,
  "p97_5": 14,
  "p99": 25
},
"errors": 0,
"timeouts": 0,
"non2xx": 0,
"run": 1
},
{
  "configuration": "baseline",
  "connections": 50,
  "duration": 10,
  "requests": 18546.19,
  "latency": {
    "average": 2.18,
    "median": 2,
    "p97_5": 3,
    "p99": 4
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 2
},
{
  "configuration": "jwt",
  "connections": 50,
  "duration": 10,
  "requests": 3806.46,
  "latency": {
    "average": 12.55,
    "median": 12,
    "p97_5": 14,
    "p99": 25
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 2
},
{
  "configuration": "baseline",
  "connections": 50,
  "duration": 10,
  "requests": 18281.46,
```

```
"latency": {
  "average": 2.19,
  "median": 2,
  "p97_5": 3,
  "p99": 5
},
"errors": 0,
"timeouts": 0,
"non2xx": 0,
"run": 3
},
{
  "configuration": "jwt",
  "connections": 50,
  "duration": 10,
  "requests": 3808.1,
  "latency": {
    "average": 12.55,
    "median": 12,
    "p97_5": 14,
    "p99": 25
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 3
},
{
  "configuration": "baseline",
  "connections": 50,
  "duration": 10,
  "requests": 18410.91,
  "latency": {
    "average": 2.18,
    "median": 2,
    "p97_5": 3,
    "p99": 5
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "jwt",
  "connections": 50,
  "duration": 10,
  "requests": 3794.46,
  "latency": {
```

```
    "average": 12.57,  
    "median": 12,  
    "p97_5": 14,  
    "p99": 25  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 4  
},  
{  
  "configuration": "baseline",  
  "connections": 50,  
  "duration": 10,  
  "requests": 18528.73,  
  "latency": {  
    "average": 2.18,  
    "median": 2,  
    "p97_5": 3,  
    "p99": 4  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 5  
},  
{  
  "configuration": "jwt",  
  "connections": 50,  
  "duration": 10,  
  "requests": 3824.1,  
  "latency": {  
    "average": 12.55,  
    "median": 12,  
    "p97_5": 14,  
    "p99": 25  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 5  
},  
{  
  "configuration": "baseline",  
  "connections": 100,  
  "duration": 10,  
  "requests": 18310.55,  
  "latency": {  
    "average": 5.05,
```

```
    "median": 5,  
    "p97_5": 6,  
    "p99": 6  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 1  
},  
{  
  "configuration": "jwt",  
  "connections": 100,  
  "duration": 10,  
  "requests": 3819.37,  
  "latency": {  
    "average": 25.68,  
    "median": 25,  
    "p97_5": 28,  
    "p99": 30  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 1  
},  
{  
  "configuration": "baseline",  
  "connections": 100,  
  "duration": 10,  
  "requests": 18234.91,  
  "latency": {  
    "average": 5.07,  
    "median": 5,  
    "p97_5": 6,  
    "p99": 6  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 2  
},  
{  
  "configuration": "jwt",  
  "connections": 100,  
  "duration": 10,  
  "requests": 3834.28,  
  "latency": {  
    "average": 25.63,  
    "median": 25,
```

```
    "p97_5": 27,  
    "p99": 30  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 2  
},  
{  
  "configuration": "baseline",  
  "connections": 100,  
  "duration": 10,  
  "requests": 18297.46,  
  "latency": {  
    "average": 5.05,  
    "median": 5,  
    "p97_5": 6,  
    "p99": 7  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 3  
},  
{  
  "configuration": "jwt",  
  "connections": 100,  
  "duration": 10,  
  "requests": 3839.55,  
  "latency": {  
    "average": 25.58,  
    "median": 25,  
    "p97_5": 27,  
    "p99": 29  
  },  
  "errors": 0,  
  "timeouts": 0,  
  "non2xx": 0,  
  "run": 3  
},  
{  
  "configuration": "baseline",  
  "connections": 100,  
  "duration": 10,  
  "requests": 18154.91,  
  "latency": {  
    "average": 5.11,  
    "median": 5,  
    "p97_5": 6,
```

```
    "p99": 7
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "jwt",
  "connections": 100,
  "duration": 10,
  "requests": 3810.82,
  "latency": {
    "average": 25.77,
    "median": 25,
    "p97_5": 28,
    "p99": 50
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 4
},
{
  "configuration": "baseline",
  "connections": 100,
  "duration": 10,
  "requests": 18282.91,
  "latency": {
    "average": 5.05,
    "median": 5,
    "p97_5": 6,
    "p99": 6
  },
  "errors": 0,
  "timeouts": 0,
  "non2xx": 0,
  "run": 5
},
{
  "configuration": "jwt",
  "connections": 100,
  "duration": 10,
  "requests": 3817,
  "latency": {
    "average": 25.74,
    "median": 25,
    "p97_5": 27,
    "p99": 50
  }
}
```



```
    },  
    "errors": 0,  
    "timeouts": 0,  
    "non2xx": 0,  
    "run": 5  
  }  
]
```

Appendix C – Analysis Code

analyze.py:

```
import json

from collections import defaultdict

import statistics


def load_data(filename):

    with open(filename, "r") as file:

        return json.load(file)


def group_results(data):

    groups = defaultdict(list)

    for result in data:

        key = (

            result["configuration"],

            result["connections"]

        )

        groups[key].append(result)
```

```
return groups
```

```
def get_metric_values(results, metric):
```

```
    values = []
```

```
    for result in results:
```

```
        if metric == "throughput":
```

```
            values.append(result["requests"])
```

```
        elif metric == "latency":
```

```
            values.append(result["latency"]["average"])
```

```
        elif metric == "p99":
```

```
            values.append(result["latency"]["p99"])
```

```
    return values
```

```
def calculate_statistics(values):
```

```
mean = sum(values) / len(values)

std_dev = statistics.stdev(values)

return mean, std_dev


def analyze_metric(groups, metric):

    results = {}

    for key, group in groups.items():

        values = get_metric_values(

            group,

            metric

        )

        mean, std_dev = calculate_statistics(

            values

        )

        results[key] = {

            "values": values,
```

```
        "mean": mean,  
  
        "std_dev": std_dev  
    }  
  
    return results
```

Reference list

1. Fielding, R.T. (2000). *Architectural styles and the design of network-based software architectures*. Doctoral dissertation. University of California, Irvine. Available from <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm> [Accessed 28 September 2026].
2. Jones, M., Bradley, J. and Sakimura, N. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force. Available from <https://www.rfc-editor.org/rfc/rfc7519.html> [Accessed 28 September 2026].
3. OWASP (2023). *OWASP Top 10 API Security Risks – 2023*. OWASP API Security Project. Available from <https://api-security.owasp.org/editions/2023/en/0x11-t10/> [Accessed 28 September 2026].
4. Jones, M., Bradley, J. and Sakimura, N. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force. Available from <https://www.rfc-editor.org/rfc/rfc7519.html> [Accessed 28 September 2026].